Problem solving · Quick concepts

Interview revision sheet. Read it the evening before, then test yourself in Interview practice.

A method for any issue

- 1. Define the problem: what fails, for whom, since when, how often.
- 2. Scope impact: one user, some users, everyone; one server or all.
- 3. Ask what changed: deployments, config, certificates, data, jobs, infrastructure, vendors.
- 4. Form a hypothesis from monitoring, logs and traces.
- 5. Test it, then fix or apply a workaround. 6. Verify. 7. Document and open a problem ticket.

Patterns to recognise

- One user fails: account, role, lockout or that user's data.
- One server fails: config, disk, memory or a bad deploy on that node; remove it from traffic.
- Slow at a fixed time: peak load or overlapping batch jobs.
- Slower over days until restart: memory or connection leak.
- Works in UAT, fails in prod: config, data, versions, network rules.
- Works in Postman, fails in app: headers, auth, payload or network path.

Error clues

- 401 everywhere: expired token, secret or certificate.
- 429: rate limiting; add backoff and spread calls.
- 502: bad upstream response; 504: upstream too slow.
- SSL handshake errors after renewal: missing intermediate certificate.
- "Too many open files": leak or low ulimit.

Data issues

- Reconcile counts and totals stage by stage to find the divergence.
- Make batch jobs idempotent so reruns do not duplicate data.
- Before an UPDATE: reviewed script, SELECT check, backup, approval.

Kubernetes quick path

- describe pod (events), logs --previous (crash reason), get endpoints (service routing).

Communication

- During P1: impact, actions, workaround, next update time.
- After: blameless RCA; "human error" is never the final root cause.